Fase 2

Plataforma IoT para Monitoreo Preventivo de Fatiga y Seguridad de Trabajadores de Construcción

Sara Escobar Sánchez, Salomé Zuluaga Sánchez

Facultad de ingeniería, Universidad Autónoma de Manizales - 22/09/2026

sara.escobars@autonoma.edu.co

salome.zuluagas@autonoma.edu.co

INTRODUCCIÓN

Los trabajadores del sector de la construcción se encuentran expuestos durante sus jornadas laborales a movimientos repetitivos, esfuerzos físicos constantes y vibraciones generadas por herramientas mecánicas, el proceso de adquisición continua de información relacionada con estos movimientos puede permitir, luego de un procesamiento , identificar patrones asociados tanto al uso de herramientas como a posibles movimientos involuntarios de la enfermedad del Parkinson, que en ocasiones, en población de este tipo, suelen pasar desapercibidos

Con este proyecto se busca implementar no solo el uso de sensores para obtención de datos, si no una arquitectura completa que permita adquirir, transmitir, autenticar, almacenar y visualizar la información de manera segura, por este motivo, el proyecto se ha desarrollado basado en servicios creados de forma independiente, comunicación mediante MQTT y contenerización mediante Docker.

Durante la primera etapa se planteó una arquitectura general compuesta por dispositivos IoT, broker MQTT, backend mediante FastAPI, base de datos y dashboard, para esta segunda entrega, el trabajo se concentró en implementar de manera funcional el flujo de información y validar la comunicación entre los diferentes servicios. Como resultado se desarrolló una arquitectura conformada actualmente por seis contenedores Docker, siendo estos: el simulador de datos, Mosquitto, puente de comunicación, InfluxDB, Grafana y una API de autenticación de dispositivos.

La utilización de un simulador permitió realizar inicialmente las pruebas sin depender del dispositivo físico, los datos generados corresponden a valores de aceleración en los ejes X, Y y Z, frecuencia y diferentes estados de movimiento, estos datos son transmitidos mediante MQTT utilizando TLS, almacenados en InfluxDB y posteriormente visualizados mediante Grafana

OBJETIVOS

Objetivo general

Diseñar, implementar y desplegar una plataforma IoT que permita el enrolamiento seguro de múltiples nodos wearable, el envio y almacenamiento de su telemetría, y su visualización y control remoto a través de una red segmentada, autenticada y contenerizada.

Objetivos específicos

1. Implementar mediante Docker los servicios necesarios para la generación, comunicación, almacenamiento y visualización de los datos del sistema.
2. Configurar un broker Mosquitto utilizando el protocolo MQTT con comunicación segura mediante TLS para permitir el transporte de la información entre los diferentes componentes de la arquitectura.
3. Desarrollar un simulador capaz de generar datos correspondientes a los ejes de aceleración, frecuencia y diferentes estados de movimiento, permitiendo realizar pruebas sin depender inicialmente del dispositivo físico.
4. Implementar un puente de comunicación encargado de suscribirse al tópico MQTT y almacenar la información recibida en una base de datos InfluxDB.
5. Verificar el almacenamiento de los datos de vibración y su correcta organización como información de series temporales.
6. Configurar Grafana como herramienta de visualización y comprobar su conexión con InfluxDB mediante la representación gráfica de los datos almacenados.
7. Identificar y solucionar los principales errores de configuración, permisos, sincronización temporal y comunicación presentados durante la integración de los diferentes contenedores.
8. Implementar una API de enrolamiento y autenticación que permita registrar dispositivos, generar API Keys individuales y validar las credenciales antes de aceptar el envío de telemetría.

METODOLOGÍA

Para la segunda fecha de entrega programada del proyecto, se realizó la habilitación de todos los contenedores Docker requeridos

y la comunicación entre ellos, dejando para la última entrega el diseño e implementación del algoritmo de Machine Learning encargado de realizar la diferenciación de los movimientos generados por herramientas mecánicas y aquellos asociados a movimientos involuntarios.

En primer lugar, se creó el directorio principal del proyecto, llamado “proyecto_redes”, mediante el comando mkdir -p proyecto_redes

Este directorio se utilizó como carpeta madre para almacenar la configuración, archivos y directorios asociados a cada uno de los contenedores implementados, a partir de esta estructura se trabajó con seis contenedores principales:

- Simulador de datos, el cual fue creado con el objetivo de realizar las pruebas iniciales de comunicación y envío de información, la idea de este contenedor es simular el comportamiento del dispositivo físico, generando datos aleatorios correspondientes a los ejes del IMU y diferentes rangos de frecuencia asociados a estados como movimiento normal, temblor tipo Parkinson, taladro y perforadora.
- Mosquitto, el cual es utilizado como broker MQTT y encargado de realizar el transporte de los mensajes generados por el simulador, para la seguridad de este proyecto se implementó la comunicación mediante TLS, utilizando el puerto 8883 y certificados previamente generados.
- Puente, es el encargado de suscribirse al tópico MQTT publicado en Mosquitto, recibir la información en formato JSON y enviarla a InfluxDB.
- InfluxDB, es utilizado como base de datos de series temporales, en este contenedor se almacenan variables relacionadas con los ejes del IMU, frecuencia de vibración, dispositivo y tipo de movimiento.
- Grafana, es utilizado para realizar la visualización de los datos almacenados en InfluxDB por medio de un dashboard interactivo.
- API, encargada del enrolamiento, autenticación y gestión de los dispositivos mediante API Keys individuales, permitiendo además recibir telemetría autenticada y almacenarla en InfluxDB.

Para permitir la comunicación entre los diferentes contenedores, sin tener que hacer uso de sus IPs se utilizó una red docker, llamada red_vibraciones, la cual permite a cada contenedor consultar a otro haciendo uso del nombre determinado.

Inicialmente, para el broker MQTT se reutilizó una configuración previamente desarrollada con TLS en el laboratorio sobre esta temática , para ello, se copiaron las carpetas del directorio original donde se almacenaba, esto para obtener la información importante como certificados, configuraciones, etc. En el archivo mosquitto.conf se encontraban definidas las configuraciones correspondientes a los puertos, certificados TLS y archivo de contraseñas. El contenedor fue configurado para trabajar principalmente mediante el puerto con cifrado 8883.

En segundo lugar, para el almacenamiento de los datos se utilizó una imagen de InfluxDB y se creó un contenedor denominado basedatos, con autorización para usar el puerto 8086. La creación de este contenedor presentó algunas dificultades de permisos, por lo cual se recomienda, realizar en primera instancia el cambio de dueño a la carpeta de trabajo, ya que inicialmente se ejecuta con un usuario determinado por influxdb 1500:1500. Para realizar las pruebas de comunicación sin depender inicialmente del dispositivo físico se creó un contenedor llamado simulator, en el cual se registraron 3 archivos principales, simulator.py , requirements.txt, y dockerfile

En el archivo simulator.py se escribió el código en Python encargado de generar los datos simulados, creando valores aleatorios para los ejes ax, ay y az y se generaron diferentes rangos de frecuencia dependiendo del tipo de movimiento seleccionado, también aleatoriamente.

Para la simulación de los datos, se definieron inicialmente cuatro estados, normal, parkinson, taladro y perforadora, con el fin de validar inicialmente el flujo de datos y tener una base sobre como se debe adquitir la información para la posterior gestion con los modelos. Para simulae el movimiento tipo Parkinson se utilizaron frecuencias simuladas aproximadamente entre 4 y 6 Hz, ya que en literatura medica se ha demostrado que estas son las predominantes en dicha enfermedad, mientras que para herramientas como taladros y perforadoras se utilizaron frecuencias superiores. Cada conjunto de datos generado se organizó en formato JSON y se publico en el topico de MQTT vibration/raw. El archivo requirement.txt contiene las dependencias requerida para la comunicación MQTT y finalmente, el archivo Dockerfile utilizó una imagen basada en Python, instalo las dependencias y configuro la ejecución automática del simulador

Posteriormente se corrio el contenedor, asociadondolo a la red_vibraciones, utilizando los certificados TLS de Mosquitto y las variables correspondientes al usuario y contraseña MQTT, una vez ejecutado el contenedor, los datos generados se verificaron mediante el comando docker logs --tail 20 simulator, observando valores correspondientes a los ejes del IMU, frecuencia y estado simulado.

Despues, se creó el contenedor puente, el cual es el encargado de establecer la conexión entre MQTT e InfluxDB, en este contenedor se crearon los archivos: puente.py, requirements.txt y Dockerfile. En el archivo puente.py se realiza inicialmente una conexión con Mosquitto mediante TLS, con la finalidad de hacerla más segura, y

se suscribe al tópico vibration/raw, despues, cuando el puente recibe un mensaje, utiliza la librería json para leer la información recibida y posteriormente organiza los datos para ser enviados a InfluxDB, la información la almacena dentro de una base de datos llamada vibraciones, la cual usa como campos principales ax, ay, az, frecuencia y tipo de máquina. En el archivo requirements.txt se adjuntan las librerías de comunicación, mientras que el Dockerfile, al igual que en el simulador, utiliza Python 3.11 como imagen base e instala automáticamente las dependencias que requiere para su funcionamiento**.**

Con el fin de comprobar directamente la información almacenada se ingresó a la consola de InfluxDB mediante el comando docker exec -it basedatos influx, posteriormente se seleccionó la base de datos vibraciones con el comando USE y se realizaron consultas utilizando SELECT * FROM vibration ORDER BY time DESC LIMIT 5.

Durante esta etapa se identificó un error que produjo problemas a la hora del envío de los datos y fue una diferencia entre la hora registrada y la hora local. Al verificar la hora del sistema mediante el comando date, se observó que la máquina virtual estaba trabajando inicialmente en UTC y que además el reloj no se encontraba sincronizado mediante NTP. Por esta razón se realizaron ajustes sobre la zona horaria y se revisó el servicio de sincronización del sistema.

Finalmente se creó un contenedor para Grafana utilizando el puerto 3000 y se almacenó en una carpeta con el mismo nombre dentro de proyecto redes

Durante la primera ejecución se presentó un error debido a que Grafana no tenía permisos para escribir dentro de la carpeta seleccionada, por esta razón se modificó el propietario de la carpeta utilizando el UID 472 correspondiente al usuario utilizado por Grafana, finalmente el contenedor pudo iniciar correctamente.Para acceder a Grafana desde el navegador se utilizó la dirección IP de la máquina virtual junto con el puerto 3000, una vez dentro de Grafana se creó una nueva fuente de datos de tipo InfluxDB utilizando las siguientes configuraciones: URL: http://basedatos:8086, Database: vibraciones y Query language: InfluxQL

La conexión se verificó mediante la opción Save & test, obteniendo como resultado que el dashboard encontraba la fuente de datos y esta estaba funcionando y que se había detectado el measurement vibration. Finalmente se creó un dashboard utilizando una visualización tipo Time series, donde se utilizó el campo frequency permitiendo observar gráficamente los cambios de frecuencia generados por el simulador.

Adicionalmente, con el fin de cumplir con el requisito de enrolamiento y autenticación de dispositivos, se creó un sexto contenedor denominado api, desarrollado mediante FastAPI, este contenedor se configuró dentro de la misma red, red_vibraciones, permitiendo su comunicación con los demás servicios del proyecto y utilizando el puerto 8000 para el acceso a la aplicación, dentro de la carpeta correspondiente a la API se crearon principalmente tres archivos: api.py, requirements.txt y Dockerfile.

En el archivo requirements.txt se registraron las librerías necesarias para el funcionamiento del servicio, principalmente FastAPI, Uvicorn y Requests. Por otra parte, el Dockerfile se configuró utilizando una imagen basada en Python 3.11, instalando automáticamente las dependencias y ejecutando el servidor mediante Uvicorn, en el archivo api.py se desarrollaron los pasos requeridos para realizar el enrolamiento, autenticación y gestión de los dispositivos. Para verificar inicialmente el funcionamiento del servicio se creó una ruta GET /health, encargado de indicar si la API se encontraba activa.

Para el enrolamiento de dispositivos se implementó la ruta POST /api/v1/devices, esta es la encargada de recibir el identificador del dispositivo que se desea registrar y genera automáticamente una API Key única, para tener una medida de seguridad adicional, esta clave se muestra únicamente al momento del registro y posteriormente se almacena solamente su versión matematica, evitando mantener la credencial original en texto plano. Para almacenar la información relacionada con los dispositivos enrolados se utilizó una base de datos SQLite dentro del propio contenedor de la API, en esta base se registra información como el identificador del dispositivo, el hash de la API Key, su estado, la fecha de creación y la fecha de la última rotación de la credencial. Esta base de datos se utiliza únicamente para la gestión de dispositivos, mientras que la información correspondiente a la telemetría continúa siendo almacenada en InfluxDB.

tambien se realizo el proceso de implementación de la ruta POST /api/v1/telemetry, la cual se encarga de recibir la información proveniente de los dispositivos que se confirmen como previamente autenticados. Para poder realizar una solicitud a esta ruta, el dispositivo debe enviar su API Key mediante el encabezado Authorization: Bearer API_KEY, la API recibe esta clave, calcula su hash (o como lo mencionamos anteriormente versión matematica y lo compara con la información almacenada en la base de datos de dispositivos. Si la clave es válida y el dispositivo se encuentra activo, la telemetría es aceptada y enviada a InfluxDB. En caso contrario, la solicitud es rechazada y se genera un log que especifica el motivo, ya sea falta de autenticación o clave erronea.

Para proteger las operaciones de la API se configuró una clave de administrador almacenada como variable de entorno mediante el archivo .env, archivo el cual fue incluido dentro del .gitignore del proyecto para evitar que la credencial fuera almacenada o publicada en el repositorio de GitHub.

Finalmente, se realizaron pruebas iniciales de autenticación. Primero se envió telemetría utilizando una API Key válida, obteniendo una respuesta satisfactoria y comprobando posteriormente el almacenamiento de la información en InfluxDB. Después se repitió la solicitud sin incluir una API Key y utilizando una clave incorrecta, obteniendo en ambos casos una respuesta 401 Unauthorized

RESULTADOS

En esta segunda entrega logramos implementar y validar la arquitectura básica de adquisición, transporte, almacenamiento y visualización de datos mediante contenedores Docker, el sistema de los contenedores quedó conformado por 6 principales: el simulador de datos, Mosquitto, puente, InfluxDB, Grafana y API.

En primer lugar, se verificó que todos los contenedores se ejecutarán de forma simultánea, confirmando que cada uno de los servicios principales se encontraba activo y disponible dentro del proyecto, esta verificación permitió comprobar que el entorno de trabajo se encontraba desplegado y en modo up y que los servicios necesarios podían ejecutarse de manera independiente.

```
salome@salomeserver:~/proyecto_redes$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS                  PORTS                                                
```

por el simulador. La visualización confirmó que los datos almacenados en InfluxDB podían ser consultados correctamente por Grafana y representados gráficamente en función del tiempo.

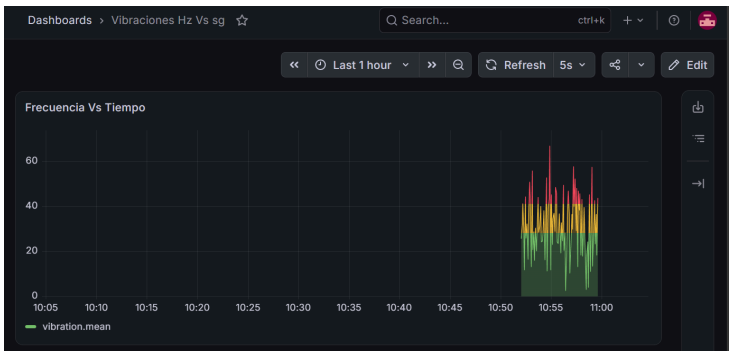


Figura 5. *Visualización temporal de la variable de frecuencia almacenada en InfluxDB mediante Grafana.*

Adicionalmente, se implementó un sexto contenedor correspondiente a la API de enrolamiento y autenticación de dispositivos, desarrollada mediante FastAPI. Una vez iniciado el servicio se verificó su funcionamiento mediante la documentación automática generada por Swagger, donde se visualizaron los diferentes endpoints disponibles para el registro, consulta, rotación y revocación de dispositivos, así como para el envío de telemetría autenticada.

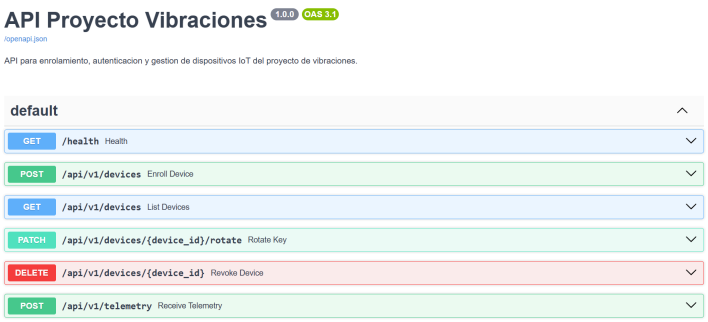


Figura 6. *Documentación automática de los endpoints de la API de enrolamiento y autenticación mediante Swagger.*

Se realizó tambie el enrolamiento del dispositivo simulado esp32_sim_01 mediante POST /api/v1/devices. Como resultado, el sistema registró correctamente el dispositivo y generó una API Key única asociada al mismo. El funcionamiento completo de este flujo puede confirmarse realizando nuevamente la consulta a la base de datos InfluxDB, para ello, se utilizó un estado de prueba denominado prueba_api, lo que permitió identificar fácilmente el registro enviado desde la API. La consulta confirmó la presencia del dato en el measurement vibration, demostrando que la telemetría autenticada era recibida por la API y posteriormente almacenada correctamente en InfluxDB, como punto adicional y con el fin de demostrar que no es posible acceder a la información sin la key correcta se realizaron pruebas utilizando una solicitud sin API Key y otra utilizando una clave incorrecta. En ambos casos la API rechazó el envío y devolvió una respuesta 401 Unauthorized.

AVANCE DEL PROYECTO Y ACTIVIDADES PENDIENTES

De acuerdo con el cronograma planteado inicialmente, para esta segunda fase se esperaba contar con la implementación de la API de enrolamiento, el primer nodo autenticado, el despliegue inicial de los servicios mediante Docker, el envío de datos hacia una base de datos y una primera evaluación de seguridad, actividades las cuales fueron desarrolladas durante la presente etapa, alcanzando aproximadamente el 65 % del proyecto final.

Hasta el momento se encuentra implementada la arquitectura principal mediante los seis contenedores Docker mencionados a lo largo del informe, todos estos servicios pueden ejecutarse simultáneamente y comunicarse mediante la red interna , también se logró implementar la comunicación mediante MQTT utilizando Mosquitto y TLS sobre el puerto 8883, además el simulador cumple tambien con su función correctamente, generando actualmente información correspondiente a aceleración en los ejes X, Y y Z, frecuencia y estado de movimiento, y publicando estos datos en el tópico vibration/raw

Para la visualización de los datos se configuró Grafana y se estableció correctamente la conexión con InfluxDB, en lo avanzado hasta el momento es posible representar gráficamente la frecuencia almacenada en la base de datos, demostrando el funcionamiento completo del flujo desde la generación de la información hasta su visualización, adicionalmente, se implementó la API de enrolamiento de dispositivos.

Actividades pendientes

Para continuar con el desarrollo del proyecto, la siguiente actividad corresponde al escalamiento de la plataforma para trabajar con un mínimo de tres nodos independientes. Cada dispositivo deberá enrolarse mediante la API y recibir una API Key diferente, permitiendo comprobar que la arquitectura puede administrar múltiples dispositivos simultáneamente. También se deberá sustituir progresivamente la información generada por el simulador por datos obtenidos desde el prototipo físico basado en ESP32-S3 y la unidad de medición inercial. Esta integración permitirá iniciar la adquisición de señales reales y establecer un conjunto de datos más representativo de los diferentes tipos de movimiento, otra de las actividades principales pendientes corresponde al desarrollo del algoritmo de Machine Learning encargado de realizar la clasificación automática de las señales, para esta etapa será necesario adquirir y organizar los datos, realizar su procesamiento, seleccionar las características relevantes y posteriormente entrenar y evaluar el algoritmo de clasificación.

EVALUACIÓN DE RIESGOS Y DE SEGURIDAD

Durante esta segunda etapa se realizó una evaluación inicial de seguridad sobre los principales componentes implementados en la plataforma, haciendo un enfoque en la protección de la comunicación, generando autenticación obligatoria a los

dispositivos que se conectan a los contenedores, archivando las credenciales en carpetas fuera de la nube y cifrando la comunicación entre contenedores.

Para iniciar con las mejoras en seguridad se configuro la comunicación del cliente y el broker MQTT mediante TLS utilizando el puerto 8883. Esta implementación permite cifrar la información durante su transmisión y reduce el riesgo de que la telemetría pueda ser interceptada o leída directamente durante su recorrido por la red, ya que no la pasa en texto plano por la via de comunicación.

```
salome@salomeserver:~$ docker port mosquitto
1883/tcp -> 0.0.0.0:1883
1883/tcp -> [::]:1883
8883/tcp -> 0.0.0.0:8883
8883/tcp -> [::]:8883
```

Figura 7. *Configuración de Mosquitto para comunicación segura mediante TLS sobre el puerto 8883.*

Adicionalmente, se implementó un mecanismo de autenticación mediante API Keys individuales para los dispositivos enrolados. Durante las pruebas se utilizó el dispositivo esp32_sim_01, al cual se le generó una credencial única durante el proceso de enrolamiento, con el fin de verificar este mecanismo se realizaron tres pruebas, en la primera se utilizó la API key que nos otorgó directamente la VM, obteniendo una respuesta satisfactoria y comprobando posteriormente que la información había sido almacenada correctamente en InfluxDB, para la segunda, se realizó una solicitud sin incluir una API Key y, en la tercera, se utilizó una key incorrecta, en estos 2 casos la API rechazó la solicitud mediante una respuesta de no autorización, confirmando que la plataforma no acepta telemetría proveniente de dispositivos que no se encuentren correctamente autenticados.



Fig 8. *Pruebas de autenticación con Key, Key valida.*

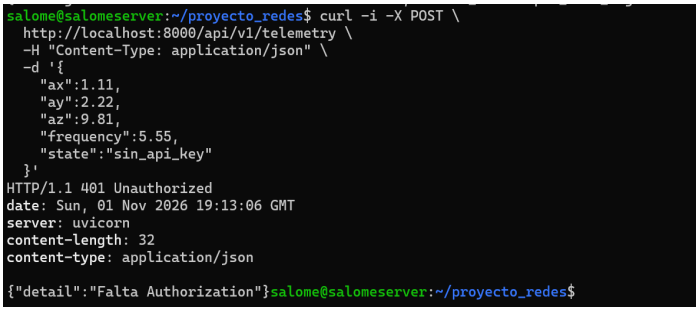


Fig 9. *Pruebas de autenticación con Key, autenticación sin key*

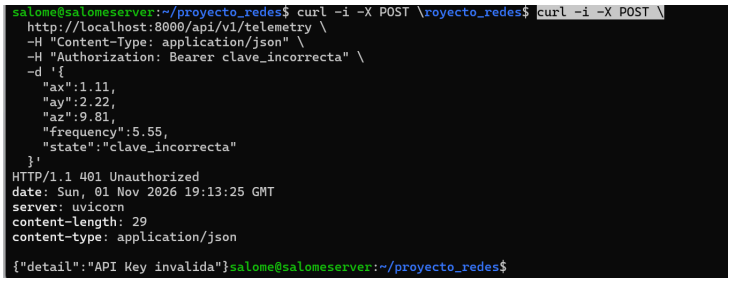


Fig 10. *Pruebas de autenticación con Key,, key invalida.*

Como medida adicional de protección, las API Keys no se almacenan directamente en texto plano dentro de la base de datos de dispositivos. La API conserva únicamente el hash de la credencial, disminuyendo el riesgo asociado a una posible exposición de la información almacenada. También se implementaron mecanismos de rotación y revocación de credenciales, permitiendo reemplazar una API Key cuando sea necesario o impedir que un dispositivo continúe utilizando la plataforma.

Para las operaciones administrativas de la API se utilizó una clave independiente almacenada mediante variables de entorno. El archivo .env fue excluido del repositorio GitHub mediante .gitignore, junto con otros archivos sensibles como las contraseñas de Mosquitto, certificados privados y directorios persistentes de los servicios.

A partir de las pruebas realizadas se identificaron los principales riesgos asociados a la arquitectura y las medidas implementadas para reducirlos. El riesgo de interceptación de la información transmitida mediante MQTT fue reducido mediante el uso de TLS, mientras que el riesgo de envío de información por dispositivos no autorizados fue mitigado mediante la autenticación con API Keys. De igual manera, la utilización de hashes y la exclusión de archivos sensibles del repositorio disminuyeron el riesgo de exposición accidental de credenciales

CONCLUSIONES

1. La utilización de MQTT mediante Mosquitto permitió realizar correctamente el transporte de la telemetría generada por el simulador, la incorporación de TLS sobre el puerto 8883 permitió agregar una capa de seguridad a la comunicación, reduciendo el riesgo de exposición de la información durante su paso por los diferentes componentes del sistema, conocer este proceso de cifrado permite realizar conexiones más seguras e implementaciones mas profesionales, ya que dejamos de lado el pensar simplemente en realizar correctamente el envio de los datos y se toma en cuenta tambien como llegan estos al destino y las fugas de información que pueden tener en el camino .
2. El uso de InfluxDB resultó adecuada para el

almacenamiento de la información producida, debido a que los datos que manejamos tiene comportamientos de serie temporal, el almacenamiento de todas las variables enviadas, como lo son la aceleración, frecuencia, dispositivo y estado permitió comprobar que el flujo de información desde el simulador hasta la base de datos funcionó de manera satisfactoria. El haber hecho una investigacón sobre que base de datos era mas conveniente para cada uno de los casos, resulto en una acertada toma de decisión sobre que adaptar para el proyecto realizado

3. La integración entre InfluxDB y Grafana permitió visualizar de forma clara el comportamiento temporal de las variables almacenadas y comprobar que la información podía ser consultada correctamente desde la base de datos, esta integración permitió entender la importancia de contar con una herramienta de visualización que facilite la interpretación de los datos y no únicamente su almacenamiento. Además, deja una base funcional para ampliar el dashboard con gráficos más completos, indicadores y, posteriormente, resultados obtenidos mediante técnicas de Machine Learning.

REFERENCIAS

[1] Docker, Inc., “Docker Documentation,” 2026.

[2] Eclipse Foundation, “Eclipse Mosquitto Documentation,” 2026.

[3] OASIS, “MQTT Version 5.0,” OASIS Standard, Mar. 7, 2019.

[4] InfluxData, “InfluxDB Documentation,” 2026.

[5] Grafana Labs, “InfluxDB Data Source,” Grafana Documentation, 2026.

[6] Grafana Labs, “Data Sources,” Grafana Documentation, 2026.

[7] Python Software Foundation, “sqlite3 — DB-API 2.0 Interface for SQLite Databases,” Python Documentation, 2026.

[8] FastAPI, “FastAPI Documentation,” 2026.

[9] OWASP Foundation, “OWASP API Security Top 10 – 2023,” 2023.

[10] M. Fagan *et al.*, “Foundational Cybersecurity Activities for IoT Product Manufacturers,” NIST IR 8259 Rev. 1, National Institute of Standards and Technology, 2026.